Check point of chapter 6
name:Dawod Andeshmand
Department: SE

6.1.1 What are the benefits of using a method?

Answer:

1. code reusability: we can use the same code many time without rewriting it.
2. Reduces code: it makes the problem shorter and simpler.

6.2.1 How do you simplify the **max** method in Listing 6.1 using the conditional operator?

Answer:

???????/

6.2.2 Define the terms parameter, argument, and method signature.

Answer: (a) parameter is the variable that can be inside of parenthesis when the method has been defined and receive the input.

Example:

```
public class Main {
    public static int sum(int a, int b) {
        return a + b;
    }
}
```

there a and b are parameter.

(b) argument is the actual amount that it sent to when calling a method.

Example: sum(10, 20); there 10 and 20 are arguments.

(c) the signature is includes the name of method, number, type and arrangement of it's parameter.

Example: public static int sum(int a, int b);

there sum(int, int) is the signature that sum is the name of method and int, int are tow parameter the type of int.

چال وطنی:

parameter = جای دریافت مقدار
argument = خود مقدار
signature = مشخصات میتو

6.3.1 How do you define a method? How do you invoke a method?
answer:
method definition: method is a block of code used to perform a specific task.
Example:

```
public class Main {
   public static void mainn(String[]args) {
  }
}
```

there mainn is a method.
A method invoke: is calling the method by using it's name parenthesis to execute it.
Example:

```
public class Main {
 public static void main(String[]args) {
 sayhello();
 }
 public static void sayhello() {
 System.out.println("hello world ");
 }
}
```

6.3.2 Reformat the following program according to the programming style and documentation guidelines proposed in Section 1.9, Programming Style and Documentation. Use the end-of-line brace style.

```
public class Main
{
   public static void main(String[]args)
   {
      sayhello();
   }
         public static void sayhello() {
         System.out.println("hello world ");

  }
}
```

end-of-line brace style:

```
public class Main{
  public static void main(String[]args){
 sayhello();
 }
  public static void sayhello() {
    System.out.println("hello world ");
    }
}
```

6.5.1 How is an argument passed to a method? Can the argument have the same name as its parameter?

(a) An argument passed to a method through the invoke of method using parenthesis, the amount of argument is transmitted to method parameter.

(b) yes an argument can have the same as it's parameter because they are in different spaces.

Example:

```
public class Main{
   public static void main(String[]args) {
      int number = 20;
      display(number);
   }
   public static void display(int number) {
      System.out.println(number);
  }
}
```

there both of name are number but first name is variable's inside of method of main and other is parameter of display's method.

6.5.2 Identify and correct the errors in the following program:

```
1  public class Test {
2    public static void main(String[] args) {
3      nPrintln(5, "Welcome to Java!");
4    }
5
6    public static void nPrintln(String message, int n) {
7      int n = 1;
8      for (int i = 0; i < n; i++)
9        System.out.println(message);
10   }
11 }
```

Answer:there are tow errors
first mistake is in the invoke of method, arrangement argument.
In the definition of method first is string and then is int;
but in the calling first is int and then is string.
Second the parameter n is declared again inside the method.
True result:

```
public class Main {
 public static void main(String[]args) {
 nPrintln("welcome to java!", 5);
 }
 public static void nPrintln(String message, int n) {
 for (int i=0; i<n; i++) {
 System.out.println(message);
 }
 }
}
```

6.5.3 What is pass-by-value? Show the result of the following programs.
Answer: the pass-by-value: when we sent an value to method, a copy of that value is given to the parameter, not the main variable itself.
Example:
int x = 10;
method(x);
if to inside of method we change the parameter's value to 20 the main value of x remains 10.
the 6.5.3 and 6.5.4 ????????/??????////??????

6.6.1 Trace the **gcd** method to find the return value for **gcd(4, 6)**.
GCD stand for Greatest Common Divisor.

```
public class Main {
   public static int gcd(int n1, int n2) {
      int gcd = 1;
      int k = 2;
      while (k<=n1 && k<=n2) {
         if (n1%k==0 && n2%k==0)
            gcd = k;
         k++;
      }
      return gcd;
   }
}
```

????
6.6.2 Trace the **isPrime** method to find the return value for **isPrime(25)**.
First what is prime number?

The prime number is the number that can only be divided into one and itself for example 2, 3, 5, 7,…..

```
public class Main {
    public static boolean isPrime(int number) {
        for (int divisor = 2; divisor <= number /2; divisor++) {
            if (number % divisor == 0)
                return false;
        }
        return true;
    }
}
```

6.7.1 What is **hexCharToDecimal('B'))**?
What is **hexCharToDecimal('7'))**?
What is **hexToDecimal("A9"))**?

Answer:

```
public class Main {
    public static void main(String[]args) {
        System.out.println(hexCharToDecimal('B'));
        System.out.println(hexCharToDecimal('7'));
        System.out.println(hexCharToDecimal("A9"));
    }
    public static int
    hexCharToDecimal(char ch) {
        if (ch >= 'A' && ch <= 'F')
            return ch - 'A' + 10;
        else
            return ch - '0';
    }
    public static int
    hexCharToDecimal(String hex) {
        int decimal =0;
        for (int i = 0; i<hex.length(); i++) {
            decimal = decimal * 16 + hexCharToDecimal(hex.charAt(i));
        }
        return decimal;
    }
}
```

output:
11
7
169

6.8.1 What is method overloading? Is it permissible to define two methods that have the same name but different parameter types? Is it permissible to define two methods

in a class that have identical method names and parameter lists, but different return value types or different modifiers?

Answer: (a) method overloading is in a class, we have the same name, but have different parameters.

Example:

```
public class Main {
    public static int sum(int a, int b) {
      return a + b;
    }
    public static double sum(double a, double b) {
        return a + b;
    }
    public static void main(String[]args) {
        System.out.println(sum(5,10));
        System.out.println(sum(2.5, 3.5));
    }
}
```

there sum and sum are tow same method but their parameter are different first parameter is int but second parameter is double.
Output: 15 and 6.0.
(b) Is it permissible to define two methods that have the same name but different parameter types?
Answer: yes, it is permissible to define two methods that have the same name but have different parameters.
Example:

```
public class Main{
    public static void show(int x) {
        System.out.println(x);
    }
    public static void show(double x) {

    }
}
```

the is sum name both tow method but have different parameter.
(c) Is it permissible to define two methods in a class that have identical method names and parameter lists, but different return value types or different modifiers?
Answer: no, tow methods con not have identical method name and parameter lists with only different return types of modifiers.
Example:

```
public class Main {
    public static int sum(int a, int b) {
        return a + b;
    }
```

```
    public static double sum(int a, int b) {
        return a + b;
    }
}
```

it is not true.

6.8.2 What is wrong in the following program?

```
public class Test {
public static void method(int x) {
  }
public static int method(int y) {

return y;
   }
}
```

there are tow method that only their parameter's name are different but is not important so java con not identify that they are different.
The correct form of that code is below

```
public class Main {
    public static void method(int x) {
        System.out.println(x);
    }
    public static int method(double y) {
        return (int)y;
    }
    public static void main(String[]args) {
        method(10);
        System.out.println(method(5.5));
    }
}
```

output= 10 and 5.

6.8.3 Given two method definitions,

public static double m(**double** x, **double** y)
public static double m(**int** x, **double** y)

tell which of the two methods is invoked for:

a. **double** z = m(**4**, **5**);
b. **double** z = m(**4**, **5.4**);
c. **double** z = m(**4.5**, **5.4**);

answer:

a. second method will be selected.
b. second method will be selected.
c. first method will be selected.

6.9.1 What is a local variable?

Answer: a local variable is a variable declared inside a method or a block of code.

Example:

```
public class Main {
   public static void main(String[]args) {
      int age = 20;
      System.out.println(age);
   }
}
```

there age is a local variable because it is inside of the main method.

6.9.2 What is the scope of a local variable?

Answer: the scope of a local variable is the part of the program where the variable can be accessed and used.

Answer:

```
public class Main {
   public static void main(String[]args) {
      int age = 20;
      if (age>=18) {
         System.out.println(age);
      }
   }

}
```

there age is defined inside the method therefore inside the is also accessed .

End!